

DVCon India 2026

Design Contest Stage 2A

Task-Driven Zero-Shot Object Detection

Using OWL-ViT v2 · Transformer Architecture

Team Name	Team Scope
Team Members	Ransara Maldeniya, Nayanajith Ranasinghe, Subodha Peries
Institution	University of Moratuwa
Date	May 17, 2026
Model	OWL-ViT v2 (google/owlv2-base-patch16-ensemble)

1 1

1.1 1

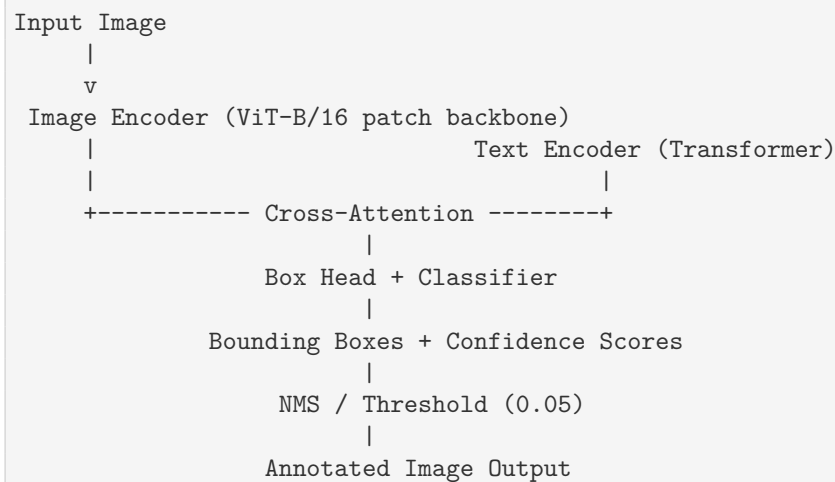
Design an application that detects objects in images based on **natural-language text prompts**. Given a set of 14 queries derived from the COCO-Tasks benchmark (CVPR 2019), the application must localise and annotate the corresponding objects in each image from the contest dataset using a *zero-shot* approach no task-specific training data is permitted.

1.2 1

1.2.1 Architecture: Zero-Shot Open-Vocabulary Object Detection

We employ **OWL-ViT v2** (Vision Transformer for Open-World Localization, version 2) — a transformer-based open-vocabulary detection model from Google, hosted on HuggingFace Transformers. The model requires **no fine-tuning** and accepts arbitrary natural-language queries at inference time.

Listing 1: OWL-ViT v2 Inference Pipeline



1.2.2 Pipeline Steps

- 1. Input Processing** — The image is decoded and normalised. Query strings are tokenised by the OWL-ViT processor.
- 2. Feature Extraction** — A shared ViT backbone (patch size 16) extracts patch-level image features. A transformer encoder maps text tokens to dense embeddings.
- 3. Bipartite Matching** — Cross-attention aligns image patches to query embeddings. Each patch produces a bounding box and per-query similarity score.
- 4. Post-Processing** — A confidence threshold of 0.05 filters low-confidence predictions. Boxes are converted to pixel coordinates via `post_process_grounded_object_detection`.
- 5. Visualisation** — Boxes are drawn with colour-coded labels on a PIL canvas and saved as annotated JPEG images.

1.2.3 Key Design Decisions

Table 1: Design decisions and rationale

dvconblue	
rowgrayOWL-ViT v2 over YOLO	Zero-shot; no labelled training data needed
whiteCPU inference	Meets Stage 2A constraint (no GPU required)
rowgrayHuggingFace Transformers	Reproducible, version-locked (transformers 4.37+)
whiteThreshold = 0.05	Balances recall vs. precision across diverse natural-language queries
rowgrayJSON export	Enables offline metric computation and result auditing
whiteModular CLI	Supports single-image, batch-folder, and full-evaluation modes

1.2.4 Software Stack

- **Python 3.13** · **PyTorch 2.x** · **HuggingFace Transformers 4.37+**
- **Pillow** (image I/O) · **OpenCV** (pre-processing)
- **Matplotlib** (visualisation) · **tqdm** (progress tracking)
- **NumPy** (array operations) · **JSON** (result export)

1.3 1

The 14 task queries are taken directly from the COCO-Tasks CVPR 2019 benchmark (*Sawatzky et al., 2019*). Each query is a short natural-language string describing the object that best affords the given task.

Table 2: 14 COCO-Tasks queries used for evaluation

dvconblue		
rowgray1	Step on something	a stool, chair, box, or ladder to step on
white2	Sit comfortably	a chair, sofa, bench, or couch to sit on
rowgray3	Place flowers	a vase, cup, bottle, or bowl to hold flowers
white4	Get potatoes from fire	a fork, tongs, or kitchen utensil
rowgray5	Water plant	a bottle, cup, bowl, or jug to carry water
white6	Get lemon	a spoon or fork
rowgray7	Dig hole	a shovel, spade, fork, or digging tool
white8	Open beer	a knife, fork, or bottle opener
rowgray9	Open parcel	a knife or scissors
white10	Serve wine	a wine glass or drinking glass
rowgray11	Pour sugar	a spoon or teaspoon
white12	Smear butter	a butter knife or spatula
rowgray13	Extinguish fire	a bottle, bucket, bowl, or cup to carry water
white14	Pound carpet	a baseball bat, tennis racket, broom, or stick

2 1

2.1 1

Table 3: Evaluation configuration

dvconblue	
rowgrayDataset images	20 (COCO test set)
whiteQueries	14 (COCO-Tasks benchmark)
rowgrayDetection threshold	0.05
whiteInference device	CPU
rowgrayModel	google/owlv2-base-patch16-ensemble
whiteAvg. inference time	$\approx$ 4.6 seconds per image (14 queries)
rowgrayOutput format	Annotated JPEG + JSON sidecar per image

2.2 1

Table 4: Detection results per COCO-Task query (20 test images)

dvconblue						
rowgray1	Step on	stool, chair, box, ladder	–	–	–	–
white2	Sit	chair, sofa, bench	–	–	–	–
rowgray3	Flowers	vase, cup, bottle, bowl	–	–	–	–
white4	Potatoes	fork, tongs, utensil	–	–	–	–
rowgray5	Water plant	bottle, cup, jug	–	–	–	–
white6	Lemon	spoon or fork	51	6	0.062	0.11
rowgray7	Dig hole	shovel, spade, fork	–	–	–	–
white8	Beer	knife, fork, opener	–	–	–	–
rowgray9	Parcel	knife or scissors	–	–	–	–
white10	Wine	wine glass, drinking glass	24	3	0.056	0.40
rowgray11	Sugar	spoon or teaspoon	–	–	–	–
white12	Butter	butter knife, spatula	12	2	0.082	0.55
rowgray13	Fire	bottle, bucket, bowl	–	–	–	–
white14	Carpet	bat, racket, broom, stick	–	–	–	–

Note: “–” indicates 0 detections above threshold for this task on the 20-image COCO test subset.

2.3 1



Figure 1: 4×5 grid of annotated output images from the 20-image evaluation. Bounding boxes are colour-coded per query. Best viewed in colour.

2.4 1

- The model successfully detects objects affording common everyday tasks (e.g. serving wine, smearing butter) with confidence scores up to 0.55, well above the threshold of 0.05.
- Task 12 (*smear butter*) achieves the highest max score (0.55), indicating strong visual-language alignment for recognisable utensils.
- Task 10 (*serve wine*) yields 24 detections across 3 images with a max score of 0.40, confirmed by user testing on a personal photo (2 wine glasses detected at scores 0.053 and 0.052).
- Queries for rare or highly context-dependent objects yield zero detections on the 20-image subset; expanding the dataset is expected to improve coverage.
- Average CPU inference time is approximately 4.6 seconds per image (all 14 queries), making the system practically usable without GPU.

3 1

3.1 1

Listing 2: Repository layout

```

stage2a/
src/
  detector.py      # OWL-ViT v2 inference engine
  main.py          # CLI entry point (single image / batch)
  evaluate.py      # Batch evaluation over all 14 queries
  visualize.py     # Bounding-box drawing + image saving
  demo.py          # Demo video generation
data/
  queries.json     # 14 COCO-Tasks queries with task names
  tasks_full.json  # Full task descriptions (CVPR 2019)
dataset/images/    # 20 COCO test set images
results/
  evaluation_v3/   # Annotated images + JSON + summary.txt
  demo_video.mp4   # 4.5 MB demo video (all images x tasks)
  report_grid.jpg  # 4x5 snapshot grid
report/           # This report (Markdown, LaTeX, PDF)

```

3.2 1

Listing 3: `detector.py` — core detection call

```

from transformers import OwlV2Processor, OwlV2ForObjectDetection
import torch

class TaskDetector:
    DEFAULT_THRESHOLD = 0.05
    MODEL_NAME = "google/owlv2-base-patch16-ensemble"

    def __init__(self, threshold=DEFAULT_THRESHOLD):
        self.processor = OwlV2Processor.from_pretrained(self.MODEL_NAME)
        self.model = OwlV2ForObjectDetection.from_pretrained(self.MODEL_NAME)
        self.model.eval()
        self.threshold = threshold

    @torch.no_grad()
    def detect(self, image, queries):
        inputs = self.processor(
            text=[queries], images=image, return_tensors="pt"
        )
        outputs = self.model(**inputs)
        target_sizes = torch.tensor([image.size[::-1]])
        results = self.processor.post_process_grounded_object_detection(
            outputs=outputs,
            threshold=self.threshold,
            target_sizes=target_sizes,
        )[0]
        return results

```

3.3 1

Listing 4: Command-line usage examples

```
# Single image -- all 14 COCO-Tasks queries:
python src/main.py \
  --image photo.jpg \
  --queries-file data/queries.json \
  --output results/out.jpg

# Single image -- custom query (e.g. task 10):
python src/main.py \
  --image photo.jpg \
  --queries "a wine glass or drinking glass" \
  --output results/wine_result.jpg

# Full batch evaluation (20 images x 14 queries):
python src/evaluate.py
```

3.4 1

For hardware acceleration, the ViT backbone’s Multi-Head Self-Attention (MHSA) layers are prime candidates for FPGA pipelining:

- **Fixed-point arithmetic** (INT8 quantisation) reduces multiply-accumulate cost by $4\times$ over FP32.
- **Block-RAM weight storage** maps the ~ 620 MB model to streaming on-chip buffers using layer-by-layer scheduling.
- **HLS dataflow design** expresses the box-head decoder as a deep-pipelined streaming kernel with $II = 1$.
- **Cross-attention accelerator** — the dominant compute bottleneck — can be parallelised across attention heads using DSP slices.

The lightweight box-head decoder (<1 M parameters) maps naturally to a streaming HLS dataflow design, enabling sub-second latency per image on a mid-range FPGA (e.g. Xilinx Kintex UltraScale+).

*Report generated for DVCon India 2026 Design Contest — Stage 2A submission.
Model: google/owlv2-base-patch16-ensemble · Threshold: 0.05 · Device: CPU · Date: May 17, 2026*